DevOps Question Bank
GCP, Azure & DevSecOps

32 real interview questions - de-duplicated, topic-wise, Basic / Advanced / Scenario-Based

Every answer: plain-English idea · technical answer · example · what to say in the interview

By Priyanka Gupta — 25 September 2026

How to use this question bank

Each question has four parts:

In simple words (green box) - a plain-English analogy. Read this first; if you understand it, the technical part becomes easy to remember.

Detailed answer + Example - the real technical answer, with commands / YAML / code you can show.

What to say to the interviewer (blue box) - 2-3 lines to say out loud. Practise these aloud.

Questions that were asked more than once across the lists were merged into one (the other wordings are shown in grey as 'Also asked as'). Language-specific (Python/PHP) and non-DevOps questions were removed. Where a pasted 'model answer' was wrong, the corrected version is given.

Suggested order: all Basic questions first, then Scenario-Based (most interviews are troubleshooting-heavy), then Advanced. Use the PDF bookmarks panel to jump to any question.

Topic	Basic	Advanced	Scenario	Total
GCP, Azure & DevSecOps	12	12	8	32

Questions in this topic

Basic **5**

Q1. GCP: What is Cloud Armor, and how does it protect applications? . . . 5

Q2. GCP: What network services are available in GCP? . . . 5

Q3. GCP: What are the GCP security services other than Cloud Armor and VPC? . . . 6

Q4. GCP: What is the GCP equivalent of CloudFormation? . . . 7

Q5. GCP: What is the CloudWatch equivalent in GCP? . . . 8

Q6. GCP: What is a CDN, and how does GCP Cloud CDN work? . . . 9

Q7. GCP: What is the difference between a public cluster and a private cluster in GKE? . . . 9

Q8. Security: How do you manage secrets securely? Are Kubernetes Secrets encrypted? . . . 10

Q9. Security: What methods do you use to check code vulnerabilities? . . . 11

Q10. Security: What is the Shared Responsibility Model (in Azure, and how does it apply to AWS/GCP)? 13

Q11. Azure: What is the difference between Azure Key Vault and Managed Identity, and how do they work together? . . . 14

Q12. Azure: What is Azure Monitor Logs, and how does it differ from Azure Monitor Metrics? . . . 15

Advanced **16**

Q13. GCP: How many IP ranges are required for a VPC-native GKE cluster, and what is each range used for? . . . 16

Q14. Security: How do you secure a Kubernetes / GKE cluster at network, pod, and identity level? . . . 17

Q15. Security: How do you design IAM with least privilege and handle permissions across AWS accounts and GCP projects? . . . 18

Q16. Security: What policies and tools are used for policy compliance checks, and how do you enforce policy as code across environments? . . . 19

Q17. Security: What audit logging and encryption strategies do you follow? . . . 21

Q18. Azure: How would you use Azure Application Gateway with WAF for a sensitive banking application? 21

Q19. Azure: Explain the difference in scaling strategies for compute-intensive vs I/O-intensive workloads. 22

Q20. Security: What security measures would you implement while hosting a production web application? 24

Q21. Security: What is email/commit signing and Helm chart signing? Which tools do you use to sign Helm charts? . . . 24

Q22. Azure: How do you implement high availability for a multi-region application? . . . 25

Q23. Azure: How do you set up automated scaling for Azure App Service? . . . 26

Q24. Azure: How do you use Microsoft Defender for Cloud for workload protection? . . . 27

Scenario-Based **29**

Q25. GCP: How do you restrict and protect a web application from cyber attacks on GCP? . . . 29

Q26. Azure: During an Azure deployment you get intermittent DNS resolution issues. What can be the causes? 30

Q27. Azure: An Azure Function is being throttled. How will you detect and fix it? 31

Q28. Azure: Design a cost-optimized cloud architecture for an internal reporting app that runs every night and stores logs for 3 years. 32

Q29. Azure: The production app works fine for internal users but external users get 403 errors. How will you isolate the issue? 33

Q30. Security: A service account key/credential leaked in a public container image. How fast can you isolate and rotate it, and how do you prevent it from happening again? 34

Q31. Azure: How would you reduce cost in an over-provisioned Azure environment? 35

Q32. Security: An image passed all vulnerability scans but still got exploited in production. What did you miss? 37

GCP, Azure & DevSecOps

Basic

Q1. GCP: What is Cloud Armor, and how does it protect applications?

In simple words: Cloud Armor is the security guard standing at the front gate of your Google load balancer. It checks every visitor against a list of rules and turns away the troublemakers before they reach your house.

Detailed answer

Cloud Armor is Google Cloud's **WAF (Web Application Firewall) and DDoS protection** service. It is attached to a **backend service** behind the external Application Load Balancer (or Cloud CDN, and some other load balancer types), so traffic is filtered at Google's edge before it reaches your VMs or GKE pods.

What it gives you:

- **DDoS protection** at network level (always on) and application level (Layer 7 floods).
- **Preconfigured WAF rules** based on OWASP Top 10, for example SQL injection (`sqli-v33-stable`) and cross-site scripting (`xss-v33-stable`).
- **IP allow/deny lists** and **geo-blocking** (block or allow by country).
- **Rate limiting** - throttle or ban clients that send too many requests.
- **Adaptive Protection** - machine learning that detects unusual traffic and suggests rules.
- **Bot management** with reCAPTCHA integration.
- Rules can run in **preview mode** first, so you see what would be blocked without breaking users.

Example:

```
gcloud compute security-policies create web-policy --description "Prod WAF"

# Block SQL injection using Google's preconfigured OWASP rule
gcloud compute security-policies rules create 1000 --security-policy web-policy \
  --expression "evaluatePreconfiguredExpr('sqli-v33-stable')" --action deny-403

# Attach the policy to the backend service behind the load balancer
gcloud compute backend-services update web-backend --global --security-policy web-policy
```

What to say to the interviewer: *Cloud Armor is GCP's WAF and DDoS service that sits on the external HTTP(S) load balancer. I use it for OWASP rules like SQLi and XSS, IP and geo filtering, and rate limiting, and I always roll new rules out in preview mode first and watch the logs before enforcing.*

Q2. GCP: What network services are available in GCP?

In simple words: Think of GCP networking as the roads, gates, toll booths and signboards of a city - private roads (VPC), gates to the outside (NAT, load balancers), signboards (DNS) and express highways to your office (Interconnect/VPN).

Detailed answer

Main GCP networking services:

- **VPC** - a global private network; subnets are regional. Includes firewall rules and routes.
- **Shared VPC** and **VPC Network Peering** - share or connect networks between projects.
- **Cloud Load Balancing** - global external Application LB, regional/internal LBs, Network (TCP/UDP) LB.
- **Cloud NAT** - lets private VMs reach the internet without public IPs.
- **Cloud DNS** - managed public and private DNS zones.
- **Cloud CDN** - caches content at Google's edge.
- **Cloud VPN** and **Cloud Interconnect** - connect on-prem to GCP (encrypted tunnel vs dedicated fibre).
- **Cloud Router** - dynamic BGP routing for VPN/Interconnect.
- **Private Service Connect / Private Google Access** - reach Google APIs and services privately.
- **Cloud Armor** - WAF/DDoS; **Network Connectivity Center** - hub-and-spoke connectivity.

Need	GCP	AWS	Azure
Private network	VPC (global)	VPC (regional)	VNet
Outbound for private hosts	Cloud NAT	NAT Gateway	NAT Gateway
DNS	Cloud DNS	Route 53	Azure DNS
CDN	Cloud CDN	CloudFront	Azure Front Door / CDN
Hybrid link	Interconnect / VPN	Direct Connect / VPN	ExpressRoute / VPN Gateway

Example:

```
gcloud compute networks create prod-vpc --subnet-mode custom
gcloud compute networks subnets create app-subnet --network prod-vpc \
  --region asia-south1 --range 10.10.0.0/20
gcloud compute routers create nat-router --network prod-vpc --region asia-south1
gcloud compute routers nats create nat-cfg --router nat-router --region asia-south1 \
  --auto-allocate-nat-external-ips --nat-all-subnet-ip-ranges
```

> ***What to say to the interviewer:*** *The core ones are VPC with firewall rules, Cloud Load Balancing, Cloud NAT, Cloud DNS, Cloud CDN, Cloud VPN and Interconnect with Cloud Router, plus Private Service Connect and Cloud Armor. A key difference from AWS is that a GCP VPC is global while subnets are regional.*

Q3. GCP: What are the GCP security services other than Cloud Armor and VPC?

> **In simple words:** If Cloud Armor is the gate guard, these are the ID card office, the locker room, the CCTV and the safety inspector of the building.

Detailed answer

- **Cloud IAM** - who can do what; roles, service accounts, conditions.

- **Security Command Center (SCC)** - central dashboard of misconfigurations, vulnerabilities and threats.
- **Cloud KMS / Cloud HSM** - manage encryption keys, including customer-managed keys (CMEK).
- **Secret Manager** - store and version passwords, API keys, certificates.
- **Identity-Aware Proxy (IAP)** - log in to apps and VMs (SSH) without a VPN or public IP.
- **VPC Service Controls** - a perimeter that stops data being copied out of services like BigQuery/GCS.
- **Binary Authorization** - only allow signed/approved container images to deploy on GKE.
- **Artifact Analysis** - vulnerability scanning of container images in Artifact Registry.
- **Cloud Audit Logs** - who did what, where and when.
- **Organization Policy Service** - guardrails such as "no public IPs" or "only these regions".
- **Sensitive Data Protection (Cloud DLP)**, **reCAPTCHA Enterprise**, **Certificate Manager**, **Chronicle/Google SecOps** (SIEM).

Example:

```
# Create a secret and read it
echo -n "S3cr3tP@ss" | gcloud secrets create db-password --data-file=-
gcloud secrets versions access latest --secret db-password

# Org policy: forbid external IPs on VMs across the organization
gcloud resource-manager org-policies enable-enforce \
  compute.vmExternalIpAccess --organization 123456789012
```

> ***What to say to the interviewer:*** *Beyond Cloud Armor and VPC firewalls, I would use IAM with least privilege, Secret Manager and Cloud KMS for secrets and keys, Security Command Center for posture, IAP for access without bastions, VPC Service Controls against data exfiltration, Binary Authorization for GKE, and Audit Logs plus Organization Policies for governance.*

Q4. GCP: What is the GCP equivalent of CloudFormation?

> **In simple words:** CloudFormation is AWS's own "blueprint" tool. Google had its own blueprint tool too, but now it basically hands you Terraform and runs it for you.

Detailed answer

- **Cloud Deployment Manager** was GCP's native IaC tool (YAML/Jinja/Python templates), the direct equivalent of CloudFormation. It has been **deprecated** and Google has moved customers off it.
- **Infrastructure Manager (Infra Manager)** is the current Google-managed option. It runs **Terraform** for you, stores state and keeps a history of deployments - similar to a managed Terraform service.
- **Config Connector** lets you manage GCP resources as Kubernetes objects (YAML applied to a GKE cluster).
- In practice most teams simply use **Terraform** with the `google` provider and a GCS bucket for remote state.

Cloud	Native IaC
AWS	CloudFormation (and CDK)
Azure	ARM templates / Bicep
GCP	Deployment Manager (deprecated) -> Infrastructure Manager (Terraform)

Example:

```
gcloud infra-manager deployments apply projects/my-proj/locations/us-central1/deployments/web \
  --service-account projects/my-proj/serviceAccounts/infra@my-proj.iam.gserviceaccount.com \
  --git-source-repo https://github.com/org/infra --git-source-directory envs/prod
```

> ***What to say to the interviewer:*** *The historical equivalent was Cloud Deployment Manager, which is now deprecated; Google's current managed option is Infrastructure Manager, which runs Terraform for you. In real projects I use Terraform with the google provider and a GCS backend for state.*

Q5. GCP: What is the CloudWatch equivalent in GCP?

> **In simple words:** CloudWatch is AWS's health monitor. In GCP the same job is done by a family called "Google Cloud Observability" - one part watches numbers, another keeps the diary (logs).

Detailed answer

The equivalent is **Google Cloud Observability** (formerly called Stackdriver):

- **Cloud Monitoring** - metrics, dashboards, uptime checks, alerting policies (like CloudWatch Metrics + Alarms).
- **Cloud Logging** - central logs, Log Explorer, log-based metrics, log sinks to BigQuery/GCS/Pub/Sub (like CloudWatch Logs).
- **Cloud Trace** - distributed tracing (like X-Ray).
- **Cloud Profiler** and **Error Reporting**.
- **Google Cloud Managed Service for Prometheus** - run PromQL against GKE metrics without running Prometheus storage yourself.

AWS	GCP
CloudWatch Metrics / Alarms	Cloud Monitoring / Alerting policies
CloudWatch Logs	Cloud Logging
X-Ray	Cloud Trace
CloudTrail	Cloud Audit Logs

Example:

```
# Read recent error logs from GKE containers
gcloud logging read 'resource.type="k8s_container" AND severity>=ERROR' \
  --limit 20 --freshness 1h

# Send logs older-term to a storage bucket (cheap retention)
gcloud logging sinks create archive-sink storage.googleapis.com/my-log-archive \
  --log-filter 'severity>=WARNING'
```

What to say to the interviewer: *The CloudWatch equivalent is Google Cloud Observability - Cloud Monitoring for metrics and alerts, Cloud Logging for logs, and Cloud Trace for tracing. CloudTrail's equivalent is Cloud Audit Logs.*

Q6. GCP: What is a CDN, and how does GCP Cloud CDN work?

In simple words: A CDN is like keeping copies of a popular book in libraries in every city, so readers don't all travel to the one central library. Users get the nearest copy, faster.

Detailed answer

A **CDN (Content Delivery Network)** caches content (images, JS, CSS, videos, even API responses) at edge locations close to users. Benefits: lower latency, less load on the origin, lower egress cost, and some DDoS absorption.

How **Cloud CDN** works:

1. It is enabled on a **backend service or backend bucket** of the global external Application Load Balancer.
2. A user request hits the nearest Google edge (anycast IP).
3. **Cache hit** -> served straight from the edge. **Cache miss** -> fetched from the origin (GCS bucket, VMs, GKE, or an external origin), then cached.
4. Caching follows `Cache-Control` headers or a **cache mode** (`CACHE_ALL_STATIC`, `USE_ORIGIN_HEADERS`, `FORCE_CACHE_ALL`) with a TTL.
5. You can **invalidate** paths after a deploy, and use **signed URLs/cookies** for private content.
6. Cloud Armor edge policies can be combined with it.

Example:

```
gcloud compute backend-buckets create static-bucket --gcs-bucket-name my-static-site \
  --enable-cdn --cache-mode CACHE_ALL_STATIC --default-ttl 3600

# After a release, clear the old files
gcloud compute url-maps invalidate-cdn-cache web-map --path "/assets/*"
```

What to say to the interviewer: *A CDN caches content at edge locations near users to cut latency and origin load. In GCP, Cloud CDN is switched on per backend service or backend bucket of the global HTTP(S) load balancer; it caches based on Cache-Control or cache mode, and I invalidate paths after each deployment.*

Q7. GCP: What is the difference between a public cluster and a private cluster in GKE?

In simple words: A public cluster is a house where every room has a door onto the street. A private cluster is a gated building: rooms have no street doors, and even the manager's office (control plane) can be reached only through the gate you choose.

Detailed answer

	Public cluster	Private cluster
Node IPs	Nodes get public (external) IPs	Nodes have only internal IPs
Control plane endpoint	Public endpoint	Private endpoint; public endpoint optional and can be restricted
Outbound internet from nodes	Directly	Needs Cloud NAT
Pulling from Google APIs/Artifact Registry	Direct	Via Private Google Access
Security	Larger attack surface	Recommended for production

Key points for private clusters:

- You give the control plane its own small range (for example `--master-ipv4-cidr 172.16.0.0/28`).
- Use **authorized networks** to allow only specific CIDRs (office VPN, CI runners) to reach the API server.
- Admin access usually via a bastion with IAP, a VPN, or the private endpoint from inside the VPC.
- Newer GKE versions describe this as "control plane access" plus "private nodes" options, but the idea is the same.

Example:

```
gcloud container clusters create prod-gke --region asia-south1 \
  --enable-ip-alias --enable-private-nodes --master-ipv4-cidr 172.16.0.0/28 \
  --enable-master-authorized-networks --master-authorized-networks 203.0.113.10/32
```

> ***What to say to the interviewer:*** *In a public GKE cluster nodes have external IPs and the API endpoint is public; in a private cluster nodes only have internal IPs, outbound goes through Cloud NAT, and the control plane is private or restricted with authorized networks. For production I always use private nodes.*

Q8. Security: How do you manage secrets securely? Are Kubernetes Secrets encrypted?

Also asked as: Kubernetes Secrets aren't encrypted - what do you use instead? Where do you keep API keys and DB passwords?

> **In simple words:** A Kubernetes Secret is like writing your password in a sealed envelope - base64 just folds the paper, anyone who opens the envelope can read it. Real security is a bank locker (Vault/KMS) with a key you control, where only specific people can open it and every access is recorded.

Detailed answer

Facts to state clearly:

- A Kubernetes Secret is only **base64-encoded, not encrypted**. `echo <value> | base64 -d` reveals it.

- By default it is stored in **etcd** without extra encryption unless you enable **encryption at rest** (EncryptionConfiguration with a KMS provider; on EKS/GKE/AKS this is "envelope encryption with KMS").
- Anyone with `get secrets` RBAC permission in the namespace can read them.

Good practice:

- Keep the source of truth in a **secret manager**: HashiCorp Vault, AWS Secrets Manager / SSM Parameter Store, GCP Secret Manager, Azure Key Vault.
- Sync into pods with **External Secrets Operator** or the **Secrets Store CSI Driver**, or inject via Vault Agent.
- For GitOps, encrypt secrets in Git with **SOPS** (KMS-backed keys) or **Sealed Secrets** - never plain YAML in Git.
- Use workload identity (IRSA/EKS Pod Identity, GKE Workload Identity, Azure Workload Identity) so pods get cloud access without static keys.
- Tight RBAC, rotation, and audit logging on secret reads.

Example:

```
# ExternalSecret: pull db password from AWS Secrets Manager into a K8s Secret
apiVersion: external-secrets.io/v1beta1
kind: ExternalSecret
metadata:
  name: db-creds
spec:
  refreshInterval: 1h
  secretStoreRef:
    name: aws-secrets
    kind: ClusterSecretStore
  target:
    name: db-creds
  data:
    - secretKey: password
      remoteRef:
        key: prod/app/db
        property: password
```

> ***What to say to the interviewer:*** *Kubernetes Secrets are only base64 encoded, so I enable KMS encryption at rest for etcd, restrict secret access with RBAC, and keep the real secrets in a manager like Vault or AWS Secrets Manager synced via External Secrets Operator. In Git I only store SOPS or Sealed Secrets encrypted files, and pods get cloud access via workload identity, not static keys.*

Q9. Security: What methods do you use to check code vulnerabilities?

Also asked as: How do you do vulnerability scanning in the pipeline? How do you detect and measure security vulnerabilities for our company?

> **In simple words:** It is like checking a car at several stations - inspect the design drawings (SAST), check the spare parts you bought from others (SCA), inspect the finished car body (image scan), then take it for a test drive to see if the brakes fail (DAST).

Detailed answer

Type	What it checks	Tools
SAST (static)	Your source code for bugs like SQLi, hardcoded secrets	SonarQube, Semgrep, CodeQL, Checkmarx
SCA (dependencies)	Known CVEs in libraries you import	Snyk, OWASP Dependency-Check, Dependabot, Trivy
Secret scanning	Passwords/keys committed to Git	Gitleaks, TruffleHog, GitHub secret scanning
Container image scan	OS packages and libs inside the image	Trivy, Grype, ECR/Artifact Registry scanning
IaC scan	Misconfigured Terraform/K8s YAML	Checkov, tfsec/Trivy config, KICS
DAST (dynamic)	The running app, attacked from outside	OWASP ZAP, Burp Suite

How I wire it in CI/CD:

1. Pre-commit hooks for secrets (Gitleaks).
2. On pull request: SAST + SCA + IaC scan; results shown on the PR.
3. After image build: Trivy scan; **fail the build on CRITICAL/HIGH** with a fix available.
4. After deploy to staging: OWASP ZAP baseline scan.
5. Continuous: registry re-scans and Dependabot PRs for new CVEs.

How I **measure** it for the company (what you can't measure, you can't improve):

- **Open vulnerabilities by severity** (Critical/High/Medium/Low), trended weekly per team or service.
- **MTTR to patch** (mean time from detection to fix) and an **SLA per severity**, for example Critical within 7 days, High within 30 and Medium within 90, reported as the % fixed within SLA.
- **Coverage**: % of repos with SAST/SCA and % of running images scanned.
- **Posture scores**: Microsoft Defender for Cloud **secure score**, AWS **Security Hub** score, GCP Security Command Center findings.
- **External validation**: periodic **pen tests**, and a **bug bounty** or responsible-disclosure program.
- Monthly dashboard to leadership, so the trend is visible and owned.

Example:

```
# Fail the pipeline if the image has fixable HIGH/CRITICAL CVEs
trivy image --severity HIGH,CRITICAL --ignore-unfixed --exit-code 1 \
  123456789012.dkr.ecr.ap-south-1.amazonaws.com/app:1.4.2

# SonarQube scan with quality gate
sonar-scanner -Dsonar.projectKey=app -Dsonar.qualitygate.wait=true
```

> ***What to say to the interviewer:*** *I layer the checks: secret scanning and SAST with SonarQube on every PR, SCA with Snyk or Trivy for dependency CVEs, Checkov for IaC, Trivy image scans that fail the build on fixable critical issues, and an OWASP ZAP DAST scan in staging. Quality gates make the pipeline block rather than just report.*

Q10. Security: What is the Shared Responsibility Model (in Azure, and how does it apply to AWS/GCP)?

In simple words: Renting a flat. The landlord looks after the building, the walls and the plumbing. You look after who you give keys to and what you keep inside. The more "furnished" the flat (PaaS/SaaS), the more the landlord does, but your valuables and keys are always your job.

Detailed answer

Security is split between the cloud provider and the customer. The split moves with the service model:

Responsibility	On-prem	IaaS (VMs)	PaaS (App Service, Azure SQL)	SaaS (M365)
Physical datacenter, hosts, network	Customer	Provider	Provider	Provider
Operating system and patching	Customer	Customer	Provider	Provider
Network controls (NSG, firewall rules)	Customer	Customer	Shared	Provider
Application code and config	Customer	Customer	Customer	Provider
Identity and access (who can log in)	Customer	Customer	Customer	Customer
Data (classification, encryption choices, backups)	Customer	Customer	Customer	Customer
Devices / endpoints	Customer	Customer	Customer	Customer

Key points:

- **You always own your data, identities, access management and devices**, whatever the service.
- AWS describes it as the provider securing **"of the cloud"** and the customer securing **"in the cloud"**. GCP calls it **shared fate** and adds secure defaults and blueprints, but the idea is the same.
- Most cloud breaches are customer-side problems: a public bucket, a leaked key, no MFA or an open NSG. None of these is the provider's failure.
- For example, with managed Kubernetes (AKS/EKS/GKE) the provider runs the control plane, but you own node patching (or turning on auto-upgrade), RBAC, network policies and your images.

Example:

```
# Customer-side checks you always own: MFA/Conditional Access, public exposure, encryption
az ad signed-in-user show --query userPrincipalName
az storage account list --query "[].{name:name,publicBlob:allowBlobPublicAccess,tls:minimumTlsVersio
    n}" -o table
az security secure-score-controls list -o table
```

What to say to the interviewer: *The Shared Responsibility Model splits security between the provider, which secures the physical infrastructure and the more of the stack the more managed the service, and the customer. The customer always owns data, identities, access and endpoints; with IaaS they also own the OS and network controls, while PaaS and SaaS shift more to the provider. AWS calls it security of versus in the cloud, and GCP calls it shared fate.*

Q11. Azure: What is the difference between Azure Key Vault and Managed Identity, and how do they work together?

In simple words: Key Vault is a bank locker that holds your secrets. Managed Identity is the app's own ID card, issued and renewed by Azure automatically. The app shows its ID card at the locker, and the locker opens only the drawers that ID is allowed to open. No password is written down anywhere.

Detailed answer

	Azure Key Vault	Managed Identity
What it is	A store for **secrets, keys and certificates**	An **identity** (a service principal in Entra ID) for an Azure resource
Solves	"Where do I keep the DB password / TLS cert / encryption key?"	"How does my app log in to Azure without a stored credential?"
Credentials	Holds them, with versioning, rotation and HSM-backed keys	None to manage; Azure issues and rotates tokens
Types	Standard / Premium (HSM), Managed HSM	System-assigned (lives and dies with the resource) or user-assigned (reusable)

How they work together:

1. Enable a managed identity on the App Service, VM, Function or AKS workload (AKS uses Workload Identity).
2. Grant that identity access on the vault: the **Key Vault Secrets User** RBAC role (recommended) or a legacy access policy.
3. The app asks the local endpoint (IMDS) for a token, and Azure SDKs do this through `DefaultAzureCredential`.
4. It calls Key Vault with the token and gets the secret. No password or connection string sits in config.
5. App Service can even use **Key Vault references** in app settings: `@Microsoft.KeyVault(SecretUri=...)`.

The **AWS analogy** is an IAM role attached to EC2/Lambda (identity) plus Secrets Manager (store). In GCP it is a service account with Workload Identity plus Secret Manager.

Example:

```
az webapp identity assign -g prod-rg -n pay-api
PRINCIPAL=$(az webapp identity show -g prod-rg -n pay-api --query principalId -o tsv)
az role assignment create --assignee $PRINCIPAL --role "Key Vault Secrets User" \
  --scope $(az keyvault show -n pay-kv --query id -o tsv)
az webapp config appsettings set -g prod-rg -n pay-api --settings \
  DB_PASSWORD="@Microsoft.KeyVault(SecretUri=https://pay-kv.vault.azure.net/secrets/db-pass/)"
```

What to say to the interviewer: *Key Vault stores secrets, keys and certificates, while Managed Identity gives the app an Entra ID identity with no credentials to manage. I enable a managed identity on the app, grant it the Key Vault Secrets User role, and the app gets a token and reads the secret at runtime, so nothing sensitive is in config. It's the same pattern as an IAM role plus Secrets Manager in AWS.*

Q12. Azure: What is Azure Monitor Logs, and how does it differ from Azure Monitor Metrics?

In simple words: Metrics are the car's speedometer and fuel gauge: small numbers, updated every minute, instant to read. Logs are the detailed trip diary: rich and searchable, but bigger, slower and more expensive to keep.

Detailed answer

	Azure Monitor Metrics	Azure Monitor Logs
Data	Numeric **time series** (CPU %, requests/sec)	Rich **records**: app logs, activity logs, resource logs, traces
Storage	Time-series metrics database	**Log Analytics workspace**
Query	Metrics Explorer, simple aggregations	**KQL (Kusto Query Language)**, joins, parsing
Latency	Near real time (about 1 minute)	Usually a few minutes of ingestion delay
Retention	Platform metrics kept 93 days	Configurable: interactive 30 days to 2 years, long-term archive up to 12 years
Cost	Platform metrics are free	Paid **per GB ingested** plus retention; the main cost driver
Alerts	Metric alerts (fast, cheap)	Log search alerts (flexible, slower)

When to use which:

- **Metrics** for fast health alerts, such as CPU > 85% for 5 minutes or HTTP 5xx rate.
- **Logs** for investigation and complex conditions, such as the top failing URLs by client IP or joining app errors with deployments.
- Send resource logs to a workspace with **Diagnostic settings**. Control cost with the Basic logs tier, daily caps, sampling and archive.
- Application Insights (workspace-based) stores its telemetry in the same workspace.

Example:

```
// KQL: 5xx errors per 5 minutes from App Service HTTP logs, last 24 hours
AppServiceHTTPLogs
| where TimeGenerated > ago(24h) and ScStatus >= 500
| summarize errors = count() by bin(TimeGenerated, 5m), CsUriStem
| order by errors desc

// Slowest dependencies seen by Application Insights
AppDependencies
| summarize p95 = percentile(DurationMs, 95) by Target
| top 5 by p95
```

What to say to the interviewer: *Azure Monitor Metrics is a near-real-time time-series store for numeric data like CPU, good for fast and cheap alerts, with 93 days of retention. Azure Monitor Logs stores rich records in a Log Analytics workspace that you query with KQL, with configurable retention and pay-per-GB ingestion. I use metric alerts for health and log queries for investigation and complex alerts.*

Advanced

Q13. GCP: How many IP ranges are required for a VPC-native GKE cluster, and what is each range used for?

In simple words: Imagine a housing society. One set of numbers for the buildings (nodes), another set for the flats inside them (pods), and a third set for shared services like the club house phone numbers (Services). They must never clash.

Detailed answer

A VPC-native (alias IP) GKE cluster needs **three ranges**:

Range	Type	Used for	Example
Primary subnet range	Subnet primary	Node IPs (VM NICs), internal LBs	10.10.0.0/20
Pod range	Secondary range	Pod IPs; each node gets a slice (default /24 = up to 110 pods)	10.20.0.0/14
Service range	Secondary range	ClusterIP addresses of Kubernetes Services	10.30.0.0/20

Extra points interviewers like:

- A **private cluster** also needs a small **/28 for the control plane** (`master-ipv4-cidr`) - not from your subnet.
- Pod IPs are real VPC IPs, so pods are routable directly from the VPC and peered networks - no overlay.
- **Size the pod range carefully**: it limits max nodes. With /24 per node, a /14 pod range gives 1024 nodes. You can reduce per-node pod IPs with `--default-max-pods-per-node`.
- Ranges must not overlap with on-prem or peered VPCs.
- Extra pod ranges can be added later (discontiguous multi-Pod CIDR) if you run out.

Example:

```
gcloud compute networks subnets create gke-subnet --network prod-vpc --region asia-south1 \
  --range 10.10.0.0/20 \
  --secondary-range pods=10.20.0.0/14,services=10.30.0.0/20

gcloud container clusters create prod-gke --region asia-south1 --enable-ip-alias \
  --subnetwork gke-subnet --cluster-secondary-range-name pods \
  --services-secondary-range-name services
```

What to say to the interviewer: *Three ranges: the subnet primary range for nodes, a secondary range for pods, and a secondary range for Services; a private cluster also needs a /28 for the control*

plane. I size the pod range for future node count because each node reserves a /24 by default.

Q14. Security: How do you secure a Kubernetes / GKE cluster at network, pod, and identity level?

Also asked as: How would you secure a Kubernetes/GKE cluster?

In simple words: Securing a cluster is like securing an office building - locked outer gates (network), rules for what each employee can do inside their room (pod), and ID badges that decide which doors open (identity). Plus CCTV (audit logs) and only trusted furniture allowed in (image policy).

Detailed answer

Infrastructure / cluster level

- **Private cluster** (private nodes), control plane restricted with **authorized networks**.
- **Shielded GKE Nodes** (secure boot, integrity monitoring), Container-Optimized OS, auto-upgrades via release channels (surge or blue-green node pool upgrades - steps are in the Terraform chapter).
- Encrypt Secrets in etcd with **Cloud KMS (application-layer secrets encryption)**.

Network level

- **NetworkPolicies** with default-deny, then allow only needed flows (GKE Dataplane V2 enforces them).
- Cloud Armor + HTTPS load balancer in front; no NodePort to the internet.
- Service mesh (Istio/Cloud Service Mesh) for **mTLS** between services if needed.

Pod / workload level

- **Pod Security Standards** (Restricted): non-root, no privileged, drop capabilities, read-only root filesystem.
- Resource limits, and GKE Sandbox (gVisor) for untrusted workloads.
- **Binary Authorization**: only images signed by our CI and scanned in Artifact Registry can run.

Identity level

- **Workload Identity Federation for GKE**: pods act as a Google service account without JSON keys.
- **RBAC** with least privilege; no `cluster-admin` for humans day to day; use Google Groups for RBAC.
- Don't use the default Compute Engine service account for nodes; create a minimal node SA.

Visibility: Cloud Audit Logs, GKE security posture dashboard, Security Command Center.

Example:

```
gcloud container clusters create prod-gke --region asia-south1 \
  --enable-private-nodes --master-ipv4-cidr 172.16.0.0/28 \
  --enable-master-authorized-networks --master-authorized-networks 10.0.0.0/8 \
  --enable-shielded-nodes --workload-pool my-proj.svc.id.goog \
  --binauthz-evaluation-mode PROJECT_SINGLETON_POLICY_ENFORCE \
  --database-encryption-key projects/my-proj/locations/asia-south1/keyRings/gke/cryptoKeys/etcd \
  --service-account gke-nodes@my-proj.iam.gserviceaccount.com --enable-dataplane-v2
```

What to say to the interviewer: *I secure it in layers: private nodes with authorized networks and shielded nodes; default-deny NetworkPolicies and Cloud Armor at the edge; Pod Security Standards and Binary Authorization for workloads; and Workload Identity plus least-privilege RBAC for identity, with KMS encryption of secrets and audit logs for visibility.*

Q15. Security: How do you design IAM with least privilege and handle permissions across AWS accounts and GCP projects?

Also asked as: How do you handle permissioning across AWS/GCP projects? (Don't say "give admin to service accounts".)

In simple words: Give people keys only to the rooms they actually work in, only for as long as they need, and keep a register of every key. In a big company you also build walls between departments (accounts/projects), so a lost key opens one department, not the whole building.

Detailed answer

Principles (both clouds)

- **Least privilege**: start from zero and grant only the actions and resources needed. No ***:***, and no admin roles on service accounts.
- Humans log in with **SSO** and get temporary access. **No long-lived keys.**
- **Grant to groups, not individuals**, so joiners and leavers are handled in one place.
- **Just-in-time (JIT)** elevated access with approval and expiry, for example for prod admin.
- **Periodic access reviews**, plus unused-permission cleanup.
- **Everything in Terraform**: IAM changes go through PRs, review and audit, never console clicks.

AWS

- **AWS Organizations** with **OUs** (Security, Prod, NonProd, Sandbox) and **SCPs** as the maximum-allowed guardrail, for example deny leaving the org, deny disabling CloudTrail, or allow only approved regions.
- **IAM Identity Center permission sets** assigned per account (ReadOnly everywhere, Developer in dev, a narrow Deployer in prod).
- **Cross-account roles** with trust policies (and an ExternalId for third parties); CI assumes a deploy role in each account.
- **Permission boundaries** so teams can create roles without escalating privilege.
- **IAM Access Analyzer**: find external access, generate policies from CloudTrail activity, flag unused permissions.
- Workloads use instance profiles and IRSA/EKS Pod Identity.

GCP

- The **org -> folders -> projects** hierarchy. Grant at the **lowest level** that works, because permissions inherit downward.
- **Google Groups** as principals. **Predefined roles** first, **custom roles** when predefined ones are too broad; avoid basic roles (Owner/Editor).
- **IAM Conditions**: limit a role by resource name prefix, tag or time.

- **Service account impersonation** (`roles/iam.serviceAccountTokenCreator`) instead of downloading keys; block keys with the org policy `iam.disableServiceAccountKeyCreation`.
- **Workload Identity Federation** for CI (GitHub/GitLab OIDC) and cross-cloud (an AWS role -> GCP), plus GKE Workload Identity for pods.
- Use the **IAM Recommender** to trim excess grants.

Example:

```
# AWS SCP: deny actions outside ap-south-1 (global services exempt); CloudTrail can't be stopped
{
  "Version": "2012-10-17",
  "Statement": [
    {"Effect": "Deny", "NotAction": ["iam:*", "sts:*", "support:*"], "Resource": "*",
     "Condition": {"StringNotEquals": {"aws:RequestedRegion": "ap-south-1"}}},
    {"Effect": "Deny", "Action": ["cloudtrail:StopLogging", "cloudtrail:DeleteTrail"],
     "Resource": "*"}
  ]
}

# GCP: group gets a narrow role on one project with a condition; CI impersonates an SA
gcloud projects add-iam-policy-binding prod-app --member group:devs@corp.com \
  --role roles/storage.objectViewer \
  --condition=expression='resource.name.startsWith("projects/_/buckets/app-logs")',title=logs-only
gcloud storage ls gs://app-logs \
  --impersonate-service-account deployer@prod-app.iam.gserviceaccount.com
```

What to say to the interviewer: *In AWS I use Organizations with OUs and SCPs as guardrails, Identity Center permission sets per account, cross-account deploy roles, permission boundaries and Access Analyzer. In GCP I use the org-folder-project hierarchy with Google Groups, predefined or custom roles at the lowest level with IAM Conditions, service account impersonation instead of keys, and Workload Identity Federation for CI and cross-cloud. In both, access is just-in-time, reviewed periodically and managed entirely in Terraform.*

Q16. Security: What policies and tools are used for policy compliance checks, and how do you enforce policy as code across environments?

Also asked as: How do you enforce policy as code across environments? OPA? Sentinel? Pulumi? What are OPA/Gatekeeper, IAM policies, Organization policies and Pod Security Standards?

In simple words: Company rules like "no one enters without a badge" only work if a guard checks every time. Policy as code turns the rule book into code, and robot guards check it at three doors: when a change is planned (CI), when it enters the cluster (admission), and all the time in the cloud (guardrails).

Detailed answer

1. Pre-deploy, in CI (cheapest place to catch problems)

- **Conftest / OPA** on the Terraform plan: `terraform show -json plan.out | conftest test -`.
- **Checkov / tfsec (Trivy config)** static scans of Terraform, Helm and K8s YAML.
- **Sentinel** in Terraform Cloud/Enterprise, with levels advisory, soft-mandatory and hard-mandatory.
- **Pulumi CrossGuard** policy packs if the team uses Pulumi.

2. Admission, in the cluster

- **OPA Gatekeeper** (ConstraintTemplates written in Rego, plus Constraints) or **Kyverno** (ClusterPolicies in YAML).
- Typical rules: no privileged pods, images only from our registry, limits required, labels required.
- **Pod Security Standards** (Baseline/Restricted) through the built-in Pod Security Admission.
- Start in **audit/warn** mode (Gatekeeper `dryrun`/`warn`, Kyverno `Audit`), then switch to **enforce/deny**.

3. Runtime and cloud guardrails (catch anything that bypassed CI)

- **AWS**: SCPs in Organizations (deny wrong regions, deny disabling CloudTrail) plus **AWS Config rules** and Security Hub.
- **GCP**: **Organization Policies**, for example `iam.disableServiceAccountKeyCreation` or no external IPs.
- **Azure**: **Azure Policy** with deny/audit effects and initiatives (CIS, PCI).

How I run it across environments

- One **policy repo**, versioned (git tags) and tested with `opa test` / `kyverno test` in its own CI.
- Promote **dev -> stage -> prod**: new rules run in **warn** mode first, we fix the violations, then **enforce**.
- **Exceptions** are explicit: a label or annotation such as `policy-exempt=reason` with an **expiry date** and an owner, reviewed regularly. Never a silent bypass.
- Policy results go to dashboards, so compliance is measurable.

Example:

```
# policy/s3.rego - run with: conftest test plan.json
package main

deny[msg] {
  r := input.resource_changes[_]
  r.type == "aws_s3_bucket_public_access_block"
  r.change.after.block_public_acls == false
  msg := sprintf("%s: public ACLs must be blocked", [r.address])
}

deny[msg] {
  r := input.resource_changes[_]
  r.change.actions[_] == "create"
  not r.change.after.tags.owner
  msg := sprintf("%s: missing required tag 'owner'", [r.address])
}

# CI steps
terraform plan -out plan.out && terraform show -json plan.out > plan.json
conftest test plan.json --policy policy/
opa test policy/ -v
```

What to say to the interviewer: *I enforce policy at three points: in CI with Conftest/OPA on the Terraform plan JSON, Checkov, or Sentinel in Terraform Cloud; at cluster admission with Gatekeeper or Kyverno plus Pod Security Standards; and at runtime with SCPs and AWS Config, GCP Organization Policies or Azure Policy. The policies live in one versioned, tested repo, get promoted dev to prod in warn-then-enforce mode, and exceptions are labelled with an owner and an*

expiry date.

Q17. Security: What audit logging and encryption strategies do you follow?

In simple words: Encryption is putting valuables in a locked box - when stored and when moving in the van. Audit logs are the CCTV recording who opened which box and when, stored somewhere the thief can't delete.

Detailed answer

Encryption

- **At rest**: enable encryption for disks, databases, buckets, backups and etcd. Default provider keys are fine for most data; use **customer-managed keys (KMS / Cloud KMS / Key Vault)** for sensitive data so you control rotation and can revoke access.
- **In transit**: TLS 1.2+ everywhere - on the load balancer, and ideally end-to-end to the backend; mTLS between services with a mesh.
- **Key management**: automatic key rotation, separate keys per environment/data class, least-privilege on who can use (`kms:Decrypt`) vs manage keys.
- **Secrets**: in a secret manager, never in code or images.

Audit logs

- Turn on **AWS CloudTrail (all regions, org trail)**, **GCP Cloud Audit Logs (incl. Data Access for sensitive services)**, **Azure Activity Log**, and **Kubernetes API audit logs**.
- Ship to a **separate, locked-down log account/project**; enable immutability (S3 Object Lock, bucket retention lock).
- Set retention per compliance (e.g., 1 year hot, 7 years archive for banking).
- Alert on high-risk events: root login, IAM policy changes, security group opened to 0.0.0.0/0, KMS key deletion; feed into a SIEM (Security Hub, Sentinel, Chronicle).

Example:

```
aws cloudtrail create-trail --name org-trail --s3-bucket-name org-audit-logs \
  --is-multi-region-trail --is-organization-trail --enable-log-file-validation
aws kms enable-key-rotation --key-id alias/prod-data
```

What to say to the interviewer: *I encrypt everything at rest and in transit, using customer-managed KMS keys with rotation for sensitive data, and TLS 1.2 plus mTLS inside the mesh. All control-plane activity goes to organization-wide CloudTrail or Cloud Audit Logs stored immutably in a separate security account, with alerts on risky actions and a SIEM on top.*

Q18. Azure: How would you use Azure Application Gateway with WAF for a sensitive banking application?

In simple words: Application Gateway is the bank's front desk that decides which counter you go to; WAF is the security scanner at the door. For a bank you add metal detectors, ID checks, CCTV and a locked vault behind them.

Detailed answer

Design:

1. **Application Gateway WAF_v2 SKU**, **zone-redundant**, with autoscaling (min instances set for peak).
2. **WAF policy in Prevention mode** with the Microsoft **Default Rule Set (DRS 2.x)** / OWASP CRS, plus **Bot Manager** rules. Tune false positives with exclusions after running in Detection mode on staging.
3. **Custom rules**: geo-filter (e.g., allow only India), rate limiting, block known bad IPs, allow admin paths only from office IPs.
4. **TLS**: HTTPS only, TLS 1.2+ with a strong predefined SSL policy; certificates stored in **Azure Key Vault**, pulled by App Gateway via **managed identity**; **end-to-end TLS** to the backends.
5. **Backends are private**: App Service/AKS behind **private endpoints** or internal IPs; backend access restricted so traffic can only come through App Gateway.
6. **Network**: App Gateway in its own subnet with NSG, **Azure DDoS Network Protection** on the VNet. For global users, put **Azure Front Door Premium** (with WAF) in front.
7. **Monitoring**: WAF and access logs to **Log Analytics** and **Microsoft Sentinel**; alerts on blocked-request spikes.
8. **Compliance**: Azure Policy to enforce WAF enabled and TLS versions (PCI-DSS, RBI guidelines).

Example:

```
az network application-gateway waf-policy create -g bank-rg -n bank-waf \
  --type Microsoft_DefaultRuleSet --version 2.1
az network application-gateway waf-policy policy-setting update -g bank-rg \
  --policy-name bank-waf --mode Prevention --state Enabled
az network application-gateway waf-policy custom-rule create -g bank-rg \
  --policy-name bank-waf -n geoBlock --priority 10 --rule-type MatchRule --action Block
```

> ***What to say to the interviewer:*** *I would deploy a zone-redundant App Gateway WAF_v2 with the Microsoft Default Rule Set in Prevention mode, custom geo and rate-limit rules, TLS 1.2 end-to-end with certificates from Key Vault, and private backends reachable only through the gateway. DDoS Protection, WAF logs into Sentinel and Azure Policy for compliance complete the design.*

Q19. Azure: Explain the difference in scaling strategies for compute-intensive vs I/O-intensive workloads.

> **In simple words:** A compute-heavy job is like a kitchen short on chefs - hire more chefs or faster chefs. An I/O-heavy job is a kitchen where chefs wait for ingredients from the store - hiring chefs won't help; you need a faster supply line or a fridge nearby (cache).

Detailed answer

	Compute-intensive (video encoding, ML, reports)	I/O-intensive (DB-heavy APIs, file processing, queues)
Bottleneck	CPU/GPU	Disk, network, DB, downstream APIs
Scale signal	CPU utilization	Queue length, request latency, IOPS, connections
VM choice (Azure)	Compute-optimized (F/Fsv2), HPC, GPU (NC/ND)	Memory/storage optimized (E, L-series), Premium SSD v2 / Ultra Disk
Scale method	Scale out on CPU (VMSS autoscale, HPA), batch parallelism, Spot VMs for batch	Scale on backlog (KEDA, Functions queue triggers), cache (Azure Cache for Redis), async processing
Watch out	Noisy neighbours, CPU throttling limits in K8s	Adding instances can overload the DB; connection limits, disk throughput caps

Key points:

- For I/O workloads, **scaling out can make things worse** if the database is the bottleneck - fix with caching, read replicas, connection pooling, batching, or bigger disk throughput.
- Azure specifics: **VM Scale Sets** autoscale on CPU for compute; **Azure Batch** for big parallel compute; **Functions/Container Apps with KEDA** scaling on Service Bus/Event Hub backlog for I/O; disk performance scales with disk size/tier.

Example:

```
# KEDA scaler: scale I/O workers on Service Bus queue depth, not CPU
apiVersion: keda.sh/v1alpha1
kind: ScaledObject
metadata:
  name: order-worker
spec:
  scaleTargetRef:
    name: order-worker
  minReplicaCount: 1
  maxReplicaCount: 30
  triggers:
    - type: azure-servicebus
      metadata:
        queueName: orders
        messageCount: "50"
      authenticationRef:
        name: sb-auth
```

> ***What to say to the interviewer:*** *Compute-bound workloads scale on CPU using compute-optimized or GPU VMs, scale sets or HPA, and Spot for batch. I/O-bound workloads scale on backlog or latency with KEDA and need faster storage, caching and connection pooling, because simply adding instances can overload the database.*

Q20. Security: What security measures would you implement while hosting a production web application?

Also asked as: What are best practices to protect a hosted web application? (AWS-specific version is in the AWS chapter.)

> **In simple words:** Defense in depth - like a bank with a guard, locked doors, a vault, cameras and an alarm. If one layer fails, the next one still stops the thief.

Detailed answer

Layer by layer:

- **Edge**: CDN + **WAF** (OWASP rules, rate limiting, bot protection) + **DDoS protection**; HTTPS only with HSTS.
- **Network**: app servers in **private subnets**, only the load balancer is public; tight security groups/firewall rules; no SSH from the internet (use SSM Session Manager / IAP / Bastion).
- **Application**: input validation, parameterized queries, secure headers (CSP, X-Frame-Options), authentication with MFA/OAuth, session security, CORS limited.
- **Secrets & identity**: secrets manager, no keys in code; least-privilege IAM roles for the app.
- **Data**: encryption at rest (KMS) and in transit (TLS 1.2+); private DB endpoints; backups encrypted and tested.
- **Supply chain**: SAST/SCA/image scanning in CI, signed images, minimal non-root containers, patching.
- **Monitoring & response**: audit logs, WAF logs, SIEM alerts, runbooks, regular pentests and DR drills.

Example:

```
# Nginx security headers on the web tier
add_header Strict-Transport-Security "max-age=31536000; includeSubDomains" always;
add_header X-Content-Type-Options "nosniff" always;
add_header X-Frame-Options "DENY" always;
add_header Content-Security-Policy "default-src 'self'" always;
ssl_protocols TLSv1.2 TLSv1.3;
```

> ***What to say to the interviewer:*** *I use defense in depth: CDN with WAF and DDoS protection at the edge, private subnets with only the load balancer exposed, least-privilege IAM and a secrets manager, encryption at rest and in transit, scanned and signed images in CI, and centralized logging with alerting and regular pentests.*

Q21. Security: What is email/commit signing and Helm chart signing? Which tools do you use to sign Helm charts?

> **In simple words:** Signing is a tamper-proof seal on a medicine bottle. If the seal is intact and carries the company's stamp, you know who packed it and that nobody opened it on the way. A signature proves who made something and that it was not changed.

Detailed answer

How signing works in one line: the author signs with a **private key**; anyone can check it with the matching **public key**. If even one byte changes, verification fails.

- **Email signing**: **S/MIME** (certificate-based, common in Outlook) or **PGP/GPG** signs emails so the receiver knows the sender is real and the text was not altered.
- **Git commit signing**: sign commits and tags with **GPG**, **SSH keys** or **Sigstore gitsign** (`git commit -S`). GitHub/GitLab show a **Verified** badge, and branch protection can **require signed commits**, so nobody can push code pretending to be someone else.
- **Helm chart signing (classic)**: `helm package --sign` uses a **GPG key** and creates a `.prov` **provenance file** next to the `.tgz` (it holds the chart's hash plus signature). Users run `helm verify` or `helm install --verify` with the public keyring.
- **Helm charts in OCI registries (modern)**: charts pushed to ECR/Artifact Registry/Harbor are signed with **Sigstore cosign** (or **Notation**/Notary v2), the same way as container images; keyless signing uses the CI identity via OIDC.
- **Container image signing**: **cosign** or **Notation** in CI after the image scan passes; then **enforce** at deploy time with **Kyverno verifyImages**, **Gatekeeper + Ratify**, or **GKE Binary Authorization** - unsigned images are rejected.

Why it matters: stops **supply-chain attacks** (a tampered chart or image from a compromised registry) and proves who approved what for audits.

Example:

```
# Git: sign commits with GPG
git config --global user.signingkey 3AA5C34371567BD2
git commit -S -m "Add ingress config"

# Classic Helm provenance with GPG
helm package ./webapp --sign --key "DevOps Team" --keyring ~/.gnupg/secring.gpg
helm verify webapp-1.2.0.tgz --keyring ~/.gnupg/pubring.gpg

# OCI chart + image signed with cosign
helm push webapp-1.2.0.tgz oci://123456789012.dkr.ecr.ap-south-1.amazonaws.com/charts
cosign sign --key awskms:///alias/cosign 123456789012.dkr.ecr.ap-south-1.amazonaws.com/charts/webapp
    :1.2.0
cosign verify --key cosign.pub 123456789012.dkr.ecr.ap-south-1.amazonaws.com/app:1.4.2
```

> ***What to say to the interviewer:*** *Signing is a tamper-proof seal - a private-key signature that proves who produced an email, commit, chart or image and that it wasn't modified. I sign commits with GPG and enforce verified commits, sign classic Helm charts with helm package --sign and verify the .prov file, and for OCI charts and images I use cosign with a KMS key, enforcing only signed images at admission with Kyverno or Binary Authorization.*

Q22. Azure: How do you implement high availability for a multi-region application?

> **In simple words:** Don't keep all your shops in one mall. First spread tills across several floors (zones) so one broken escalator doesn't close the shop. Then open a second mall in another city (region) with a signboard on the highway (Front Door) that sends customers to whichever mall is open.

Detailed answer

Layer 1 - inside a region: Availability Zones

- Zone-redundant services: **App Service** (zone redundancy, 3+ instances), **AKS** node pools across zones 1/2/3, zone-redundant Application Gateway, and **ZRS** storage.
- Zones protect against one datacenter failing, with no data loss.

Layer 2 - across regions (usually Azure paired regions, e.g. Central India + South India)

- **Global entry point**:
- **Azure Front Door**: Layer 7, global anycast, WAF, TLS, caching, **health probes**, failover in seconds, path-based routing.
- **Traffic Manager**: **DNS-based**, works for any protocol, but failover depends on the DNS TTL.
- **Data tier** (the hard part):
- **Azure SQL auto-failover groups**: geo-replication with a single listener endpoint that follows the primary.
- **Cosmos DB** multi-region writes, or a single write region with automatic failover.
- **Storage**: GRS/GZRS, or **RA-GZRS** if you need to read the secondary.
- **Redis**: geo-replication (Premium/Enterprise).
- **Patterns**:
- **Active-active**: both regions serve traffic. Best RTO, but needs data consistency handling.
- **Active-passive (warm standby)**: the secondary is scaled down and scaled up on failover. Cheaper.

Make it real

- Everything is in IaC, so the second region is identical.
- Deep health endpoints are used as probes.
- Define **RTO/RPO**, write failover runbooks, and run **regular DR drills**.

Example:

```
az sql failover-group create -g prod-rg --server sql-cin --partner-server sql-sin \
  -n pay-fog --add-db paydb --failover-policy Automatic --grace-period 1
az afd origin create -g prod-rg --profile-name pay-fd --origin-group-name web \
  --origin-name cin --host-name pay-cin.azurewebsites.net --priority 1 --weight 1000 \
  --enabled-state Enabled --origin-host-header pay-cin.azurewebsites.net
az afd origin create -g prod-rg --profile-name pay-fd --origin-group-name web \
  --origin-name sin --host-name pay-sin.azurewebsites.net --priority 2 --weight 1000 \
  --enabled-state Enabled --origin-host-header pay-sin.azurewebsites.net
```

> ***What to say to the interviewer:*** *I use Availability Zones inside each region, with zone-redundant App Service or AKS and ZRS storage, and then a paired secondary region behind Azure Front Door with health probes. Front Door is my default because it gives layer 7 failover in seconds plus WAF; Traffic Manager is the option for DNS-based or non-HTTP traffic. Data uses SQL auto-failover groups, Cosmos DB multi-region or RA-GZRS storage, and I choose active-active or active-passive by the RTO/RPO and budget, then prove it with DR drills.*

Q23. Azure: How do you set up automated scaling for Azure App Service?

> **In simple words:** Scaling up is buying a bigger oven. Scaling out is adding more ovens. Autoscale is a manager who adds ovens when the orders queue grows and removes them when it's quiet, with a short wait in between so it doesn't keep switching them on and off.

Detailed answer

- **Scale up** = a bigger App Service plan tier/size (more CPU/RAM per instance). This is manual and causes a restart.
- **Scale out** = more instances of the plan. This is what autoscale does.

Autoscale (Azure Monitor autoscale settings):

- Needs **Standard tier or higher** (Premium v3 recommended); Basic/Free can't autoscale.
- **Rules** on metrics such as **CPU %**, **memory %**, **HTTP queue length**, or a Service Bus queue for workers.
- Always create a **scale-out rule and a matching scale-in rule**, with **cooldowns** (e.g. 5-10 minutes) to avoid flapping.
- Set **min/max/default instances**. Keep min >= 2 or 3 for HA (and for zone redundancy).
- **Scheduled profiles**: for example, more instances during business hours or before a known sale event.
- Autoscale settings are per **plan**, so all apps on the plan scale together.

Automatic scaling (the newer Premium v2/v3 feature): App Service scales on **HTTP traffic** with pre-warmed instances. You set a maximum burst and optional always-ready instances, with no rules to write.

Watch out: scaling the web tier doesn't help if the database is the bottleneck, and slow app startup delays scale-out, so use warm-up and health check settings.

Example:

```
az monitor autoscale create -g prod-rg --resource pay-plan \
  --resource-type Microsoft.Web/serverfarms --name pay-autoscale \
  --min-count 2 --max-count 10 --count 2

az monitor autoscale rule create -g prod-rg --autoscale-name pay-autoscale \
  --condition "CpuPercentage > 70 avg 10m" --scale out 2 --cooldown 5
az monitor autoscale rule create -g prod-rg --autoscale-name pay-autoscale \
  --condition "CpuPercentage < 30 avg 15m" --scale in 1 --cooldown 10
az monitor autoscale rule create -g prod-rg --autoscale-name pay-autoscale \
  --condition "HttpQueueLength > 100 avg 5m" --scale out 2 --cooldown 5
```

> ***What to say to the interviewer:*** *I keep the plan on Premium v3 and create autoscale settings with scale-out and scale-in rule pairs on CPU, memory or HTTP queue length, with cooldowns, a minimum of 2-3 instances for HA and a sensible maximum. I add scheduled profiles for known peaks, or use the newer automatic scaling for HTTP-driven apps. Scaling up to a bigger SKU is separate, and I check that the database won't become the next bottleneck.*

Q24. Azure: How do you use Microsoft Defender for Cloud for workload protection?

Also asked as: What is Azure Defender / Azure Security Center?

> **In simple words:** Defender for Cloud is both a building inspector and a security guard. The inspector gives you a checklist and a score for what's unlocked or misconfigured (posture). The guard watches your servers, containers and databases live and raises the alarm when someone breaks in (threat protection).

Detailed answer

Two halves:

- **CSPM (Cloud Security Posture Management)**
- **Secure score** plus prioritized **recommendations**, such as "enable MFA", "storage allows public access" or "VMs missing updates".
- **Regulatory compliance dashboards**: Microsoft Cloud Security Benchmark, CIS, PCI-DSS, ISO 27001.
- Works multi-cloud by connecting AWS and GCP accounts.
- Defender CSPM (the paid plan) adds attack path analysis and agentless scanning.
- **CWP (Cloud Workload Protection) - Defender plans per resource type**
- **Servers**: vulnerability assessment, EDR (Defender for Endpoint), file integrity monitoring, **just-in-time VM access**, which opens RDP/SSH ports only on request, for a limited time and from your IP.
- **Containers**: **image vulnerability scanning** in ACR, **runtime threat detection** on AKS (e.g. crypto-miners, privileged container launch) and Kubernetes posture checks.
- **SQL, Storage** (malware scanning, anomalous access), **Key Vault** (unusual secret access), **App Service**, **Resource Manager** and **DNS**.

How I roll it out:

1. Enable it at subscription or management-group level, via Azure Policy or Terraform so it's consistent.
2. Turn on the Defender plans for the resource types we actually run.
3. Work down the recommendations by secure-score impact; exempt with a justification where needed.
4. Stream alerts to **Microsoft Sentinel** (SIEM) or Teams through workflow automation; use continuous export to Log Analytics.
5. Track secure score and compliance percentages monthly as KPIs.

Example:

```
az security pricing create -n VirtualMachines --tier Standard
az security pricing create -n Containers --tier Standard
az security pricing create -n KeyVaults --tier Standard
az security secure-score-controls list -o table
az security assessment list --query "[?status.code=='Unhealthy'].displayName" -o tsv
```

> ***What to say to the interviewer:*** *Defender for Cloud does two jobs: CSPM, with a secure score, recommendations and regulatory compliance dashboards, and workload protection plans for servers, containers, SQL, storage and Key Vault. Those plans cover image scanning in ACR, AKS runtime threat detection and just-in-time VM access. I enable it org-wide through policy, fix recommendations by score impact, and route alerts into Sentinel.*

Scenario-Based

Q25. GCP: How do you restrict and protect a web application from cyber attacks on GCP?

In simple words: Put the app behind Google's big front gate (global load balancer), station the guard there (Cloud Armor), lock all the back doors (private backends, firewall), and keep the keys in a safe (Secret Manager, IAM).

Detailed answer

Step by step:

1. **Global external Application Load Balancer** as the only entry point, with **Google-managed SSL certificates** and an SSL policy enforcing TLS 1.2+; HTTP -> HTTPS redirect.
2. **Cloud Armor** policy on the backend: OWASP preconfigured rules (SQLi, XSS, LFI, RCE), **rate limiting / rate-based ban**, geo restriction, **Adaptive Protection** for L7 DDoS, reCAPTCHA Enterprise for bots. Start in preview mode.
3. **Private backends**: GKE private nodes or MIGs with no external IPs; firewall allows port 443/80 only from Google LB and health-check ranges (`35.191.0.0/16`, `130.211.0.0/22`).
4. **Admin access** through **Identity-Aware Proxy (IAP)** - no public SSH, no bastion with open ports.
5. **Cloud CDN** for static content to absorb traffic.
6. **Data protection**: Cloud SQL with private IP, CMEK via Cloud KMS, Secret Manager for credentials, **VPC Service Controls** around sensitive data.
7. **Supply chain**: Artifact Analysis scanning + Binary Authorization.
8. **Detect**: Security Command Center, Cloud Armor logs in Cloud Logging, alerts on spikes of 403/429.

Example:

```
# Allow LB and health-check traffic only; nothing else reaches the backends
gcloud compute firewall-rules create allow-glb --network prod-vpc --direction INGRESS \
  --allow tcp:80,tcp:443 --source-ranges 35.191.0.0/16,130.211.0.0/22 --target-tags web

# Rate-based ban: >100 req/min per IP -> banned for 10 minutes
gcloud compute security-policies rules create 900 --security-policy web-policy \
  --src-ip-ranges "*" --action rate-based-ban --rate-limit-threshold-count 100 \
  --rate-limit-threshold-interval-sec 60 --ban-duration-sec 600 \
  --conform-action allow --exceed-action deny-429 --enforce-on-key IP
```

What to say to the interviewer: *I expose the app only through the global HTTPS load balancer with Cloud Armor OWASP rules, rate-based bans and Adaptive Protection, keep backends private with firewall rules allowing only Google LB ranges, and use IAP for admin access. Data is protected with private Cloud SQL, KMS and Secret Manager, and Security Command Center plus Armor logs give detection.*

Q26. Azure: During an Azure deployment you get intermittent DNS resolution issues. What can be the causes?

In simple words: DNS is the phone book. "Sometimes it works" means some phone books are outdated, some operators are overloaded, or some calls are routed to the wrong exchange.

Detailed answer

Likely causes and checks:

1. **Custom DNS servers on the VNet** (e.g., on-prem AD DNS) that don't forward to Azure DNS **168.63.129.16** - private endpoints and private zones won't resolve consistently. Fix: conditional forwarders or **Azure DNS Private Resolver**.
2. **Private DNS zone not linked** to every VNet (hub/spoke), or duplicate zones with different records -> some VMs get public IP, others private.
3. **Stale caching / TTL**: during deployments resources are recreated (new IPs, slot swaps), and clients or intermediate resolvers keep old answers until TTL expires.
4. **One of multiple DNS servers is unhealthy** - clients rotate between them, so failures look random.
5. **AKS-specific**: CoreDNS pods overloaded or scaled too low; `ndots:5` causing many extra lookups; the known **UDP conntrack race** (fix with NodeLocal DNSCache); SNAT port exhaustion on outbound.
6. **Throttling**: Azure DNS limits queries per VM (around 1000 qps per VM to 168.63.129.16) - chatty apps hit it.
7. **NSG/firewall** blocking UDP/TCP 53 to some DNS servers, or Azure Firewall DNS proxy misconfigured.
8. **Propagation delays** after creating/changing public DNS records.

How to debug:

- Test from the failing host repeatedly: `nslookup` / `dig` against each DNS server; compare results.
- Check which DNS server the VNet/VM uses; check private zone links.
- In AKS, check CoreDNS logs and metrics.

Example:

```
for i in $(seq 1 20); do dig +short myapp.privatelink.database.windows.net; done
az network vnet show -g net-rg -n spoke-vnet --query dhcpOptions.dnsServers
az network private-dns link vnet list -g dns-rg -z privatelink.database.windows.net -o table
kubectl -n kube-system logs -l k8s-app=kube-dns --tail 50
```

What to say to the interviewer: *I would check whether custom DNS servers forward to 168.63.129.16, whether the private DNS zones are linked to every VNet, TTL caching after resources were recreated, an unhealthy DNS server in the rotation, and in AKS CoreDNS load, ndots and the conntrack race. I prove it by querying each DNS server repeatedly from the failing host and comparing answers.*

Q27. Azure: An Azure Function is being throttled. How will you detect and fix it?

Also asked as: An Azure Function has frequent timeouts. How do you troubleshoot it?

In simple words: The function is a toll booth. Throttling means too many cars are arriving - either the booth itself limits cars, or the road after it (a database or API) is jammed and sends cars back.

Detailed answer

Detect

- **Application Insights**: HTTP **429** responses, failed invocations, increased duration, dependency failures.
- **Azure Monitor metrics**: function execution count, instance count, queue length growing (for queue/Event Hub triggers).
- Host logs showing the host is at max concurrency, or downstream **429 from Cosmos DB/Storage/external API**.
- Check **plan limits**: Consumption plan has cold starts and a maximum instance count; one instance handles limited concurrency.

Fix (depending on the cause)

1. **Plan limits**: move to **Premium (Elastic Premium)** or **Flex Consumption** for pre-warmed instances, higher scale-out and VNet support; or raise `functionAppScaleLimit` if it was set low.
2. **Concurrency tuning** in `host.json`: `maxConcurrentRequests` for HTTP, `batchSize`/`maxConcurrentCalls` for queue/Service Bus - to either use capacity better or protect downstream.
3. **Downstream throttling**: add retries with exponential backoff, increase Cosmos DB RU/s or enable autoscale, cache results, batch writes.
4. **Buffer bursts**: put a queue (Service Bus/Storage Queue) in front so spikes are absorbed and processed at a steady rate.
5. **Front door rate limits**: if API Management is in front, check its rate-limit policy.
6. Reduce execution time (reuse HTTP/DB clients, avoid static-per-call connections).

If the symptom is timeouts rather than 429s:

- **functionTimeout per plan**: Consumption defaults to 5 minutes (max 10), while Premium and Dedicated default to 30 minutes and can be unbounded. Set it in `host.json`.
- **HTTP triggers have a hard limit of about 230 seconds** (the front-end load balancer idle timeout), whatever functionTimeout says. For long work, return 202 quickly and use the **Durable Functions async HTTP pattern** or hand off to a queue.
- **Cold starts** on Consumption: use Premium/Flex always-ready instances, or keep the package small.
- **Outbound connection / SNAT port exhaustion**: creating a new `HttpClient` or DB connection per call exhausts ports, so reuse static clients and use connection pooling.
- **Slow downstream**: find it with **Application Insights dependency tracking** (duration per dependency, failed calls), then add timeouts, retries and caching.

Example:

```
{
  "version": "2.0",
  "functionTimeout": "00:10:00",
  "extensions": {
    "http": { "maxConcurrentRequests": 100, "maxOutstandingRequests": 200 },
    "serviceBus": { "maxConcurrentCalls": 32 }
  },
  "retry": { "strategy": "exponentialBackoff", "maxRetryCount": 5,
             "minimumInterval": "00:00:02", "maximumInterval": "00:00:30" }
}
```

What to say to the interviewer: *I detect it in Application Insights and Azure Monitor - 429s, rising duration, growing queue length - and then identify whether the limit is the plan, host concurrency or a throttled downstream like Cosmos DB. Fixes are moving to Premium or Flex Consumption, tuning host.json concurrency, backoff retries, more downstream capacity, and a queue to buffer bursts. For timeouts I check functionTimeout for the plan, the 230-second HTTP limit (moving long work to Durable Functions), cold starts, SNAT exhaustion from not reusing HttpClient, and slow dependencies in Application Insights.*

Q28. Azure: Design a cost-optimized cloud architecture for an internal reporting app that runs every night and stores logs for 3 years.

In simple words: Don't rent a full-time taxi for a trip you take once a night. Book a cab only for the ride (serverless/spot), and put old files in a cheap basement storeroom (archive tier) instead of the expensive front shelf.

Detailed answer

Design principles: **pay only while running**, **tier storage by age**, **no always-on servers**.

Piece	Azure	AWS equivalent
Nightly trigger	Logic Apps / Function timer / Container Apps Job (cron)	EventBridge Scheduler
Compute for the job	Container Apps Job or Azure Batch on **Spot VMs**	Fargate Spot / AWS Batch on Spot / Lambda
Source data	Read replica or export, so prod DB is not loaded	RDS read replica / snapshot export
Report output	Blob Storage (Hot for 30 days)	S3 Standard
Log retention 3 yrs	Log Analytics short retention (30-90 days) + export to Blob; lifecycle Cool -> Cold -> Archive	CloudWatch 30-90 days -> S3; lifecycle to Glacier Deep Archive
Access for users	Power BI / static page with Entra ID auth	QuickSight / S3 + CloudFront

Cost levers:

1. Compute scales to **zero** between runs; Spot for batch (retry if evicted).
2. **Lifecycle management**: Hot 30 days -> Cool 90 days -> Archive until 3 years -> delete. Archive tier is a tiny fraction of hot storage cost (remember retrieval takes hours - fine for audit logs).

3. Keep **Log Analytics** interactive retention short; long-term logs as compressed files in storage, not in the expensive analytics store.
4. Run in one region with **LRS/ZRS** (not GRS) unless compliance requires otherwise; use **immutability policy** if logs are for audit.
5. Budgets and cost alerts; tag everything.

Example:

```
{
  "rules": [{
    "name": "logs-3yr",
    "enabled": true,
    "type": "Lifecycle",
    "definition": {
      "filters": { "blobTypes": ["blockBlob"], "prefixMatch": ["logs/"] },
      "actions": { "baseBlob": {
        "tierToCool":    { "daysAfterModificationGreaterThan": 30 },
        "tierToArchive": { "daysAfterModificationGreaterThan": 120 },
        "delete":        { "daysAfterModificationGreaterThan": 1095 } } }
    }
  }]
}
```

> ***What to say to the interviewer:*** *I would run the nightly job as a scheduled Container Apps Job or Batch on Spot that scales to zero, read from a replica, write reports to Blob, and keep logs short-term in Log Analytics while exporting to Blob with a lifecycle policy moving to Cool, then Archive, and deleting after 3 years. On AWS that is EventBridge plus Fargate Spot and S3 lifecycle to Glacier Deep Archive.*

Q29. Azure: The production app works fine for internal users but external users get 403 errors. How will you isolate the issue?

> **In simple words:** Staff enter through the back door and walk straight in; visitors come through the front door where the guard is turning them away. So look at the front-door guard, not the building.

Detailed answer

403 means the request **reached something that refused it** (not a network timeout). The difference between internal and external users is the **path**, so compare paths:

1. **Map both paths**: internal users may hit the private IP / internal DNS directly; external users go through **Front Door / App Gateway WAF / API Management / public DNS**.
2. **Check the WAF logs first** - the most common cause. A managed rule (e.g., SQLi false positive), **geo-filter**, bot rule, or IP rule blocking. Query `AzureDiagnostics` for `ApplicationGatewayFirewallLog` / Front Door WAF logs with `action == Block`.
3. **App Service Access Restrictions** / Azure Functions IP restrictions allowing only internal ranges, or requiring the Front Door header `X-Azure-FDID` - requests bypassing Front Door get 403.
4. **Storage/Key Vault firewall** or **private endpoint only** settings - external calls to a public endpoint get 403.
5. **Authentication**: Entra ID **Conditional Access** (trusted locations/compliant device), Easy Auth, or the app rejecting tokens from external tenants.

6. **Host header / CORS / hostname** issues: external domain not in allowed hosts, or origin checks in the app.
7. Reproduce: `curl -v` from outside (mobile hotspot or cloud shell) vs inside, compare response headers (Front Door, App Gateway and App Service return different 403 pages/headers) to pinpoint the layer.

Fix: tune/exclude the WAF rule (never just disable the WAF), correct access restriction rules, or update Conditional Access - then add a synthetic external check so it is caught earlier.

Example:

```
AzureDiagnostics
| where Category == "ApplicationGatewayFirewallLog" and action_s == "Blocked"
| summarize count() by ruleId_s, Message, clientIp_s, requestUri_s
| order by count_ desc

az webapp config access-restriction show -g prod-rg -n bank-web
```

> ***What to say to the interviewer:*** *Since internal users succeed, I compare the request paths and look at the layers only external traffic passes through - first the WAF logs on Front Door or App Gateway, then App Service access restrictions, storage or Key Vault firewalls and Conditional Access. I reproduce with curl from outside, read which layer returned the 403 from the headers, and fix that rule rather than disabling protection.*

Q30. Security: A service account key/credential leaked in a public container image. How fast can you isolate and rotate it, and how do you prevent it from happening again?

> **In simple words:** Your house key was left in a box posted on the internet. Taking the box down doesn't help - someone may already have copied the key. Change the lock immediately, check the CCTV for who came in, then stop putting keys in boxes.

Detailed answer

Golden rule: **assume the credential is compromised forever**. Deleting the image from the registry does not un-leak it - anyone could have pulled or cached it.

1. **Contain (first 5-15 minutes)**: disable or delete the leaked key right away - GCP service account key, AWS access key, or K8s service account token. Don't wait until a replacement is ready; a short outage is better than an attacker with access.
2. **Remove the exposure**: make the repository or image tag private, delete the affected tags and layers, and ask any mirror or cache to purge them.
3. **Rotate and restore**: create a new credential (ideally switch to keyless, see step 6), store it in a secret manager, and redeploy the consumers so they read it from there.
4. **Investigate**: search **Cloud Audit Logs / CloudTrail** for any use of the leaked key ID - which source IPs, which API calls, and any new resources (VMs, keys, IAM bindings, crypto miners). Check the **blast radius** of that identity's permissions and revoke anything an attacker created, such as new keys or users.
5. **Hunt for more leaks**: scan other images and the git history - `trivy image --scanners secret`, `gitleaks`, `trufflehog`. Rewrite git history if needed, but still rotate.
6. **Prevent**:

- Go keyless: **GKE Workload Identity**, **IRSA/EKS Pod Identity**, and **OIDC federation for CI** (GitHub Actions to AWS/GCP), so there is no key file to leak.
- Keep secrets out of the build: BuildKit `--mount=type=secret`, a proper `.dockerignore`, multi-stage builds.
- Add secret scanning in pre-commit and CI that blocks the push or build.
- Set the org policy `iam.disableServiceAccountKeyCreation` (and an SCP or alerts on `CreateAccessKey` in AWS).
- Least privilege, so the next leak has a small blast radius.

7. Write a blameless postmortem with owners and dates for each action.

Time	Action
0-15 min	Disable/delete key, make image private, alert security
15-60 min	New credential in secret manager, redeploy consumers
1-4 hrs	Audit log review, revoke attacker-created resources
Same day	Scan all images and repos, purge history
1-2 weeks	Keyless identity, CI secret scanning, org policy, postmortem

Example:

```
# GCP: disable then delete the leaked key
gcloud iam service-accounts keys disable KEY_ID --iam-account app@my-proj.iam.gserviceaccount.com
gcloud iam service-accounts keys delete KEY_ID --iam-account app@my-proj.iam.gserviceaccount.com
gcloud logging read 'protoPayload.authenticationInfo.serviceAccountKeyName:"KEY_ID"' --freshness 30d

# AWS: deactivate the key and see where it was used
aws iam update-access-key --user-name ci-bot --access-key-id AKIA... --status Inactive
aws cloudtrail lookup-events --lookup-attributes AttributeKey=AccessKeyId,AttributeValue=AKIA...

# Hunt for other leaks
trivy image --scanners secret myorg/app:latest
gitleaks detect --source . --log-opts="--all"

# Build without baking secrets into layers
docker build --secret id=npmrc,src=$HOME/.npmrc .   # Dockerfile: RUN --mount=type=secret,id=npmrc .
    ..
```

> ***What to say to the interviewer:*** *Within minutes I disable the leaked key without waiting for a replacement, make the image private and treat the key as permanently compromised. Then I rotate to a new credential held in a secret manager, check CloudTrail or Cloud Audit Logs for any misuse and revoke what the attacker created, and scan other images and repos. To prevent a repeat I move to keyless workload identity and OIDC in CI, use BuildKit secret mounts, block pushes with secret scanning, and disable service account key creation by org policy.*

Q31. Azure: How would you reduce cost in an over-provisioned Azure environment?

> **In simple words:** You're paying for a 10-room house but live in 3 rooms with all the lights on. Find the empty rooms, switch off lights when you leave, and sign a yearly lease for the rooms you do use to get a discount.

Detailed answer

Step by step:

1. **See the spend**:

- **Cost Management + Billing**: cost by resource group, service and tag.
- **Azure Advisor** cost recommendations.
- Enforce **tags** (owner, env, app) with Azure Policy so every cost has an owner.

2. **Delete waste (quick wins)**:

- Unattached **managed disks**, old snapshots, idle **public IPs**, orphaned **NICs**.
- Empty App Service plans, idle load balancers/gateways, forgotten test resources.

3. **Rightsize**:

- Downsize VMs and App Service plans with low CPU/memory (Advisor flags under ~5% CPU).
- Move to newer, cheaper VM series.
- Scale SQL/Cosmos RU/s to the real load.

4. **Scale with demand**:

- Autoscale App Service/VMSS; set the minimum to what HA needs, not peak.
- **AKS**: cluster autoscaler per node pool, a **Spot node pool** for batch/stateless workloads, and rightsized pod requests.

5. **Switch off when unused**: **auto-shutdown** or start/stop schedules for dev/test VMs, and scale non-prod AKS to zero at night.

6. **Commit for steady usage**: **Reservations** (1 or 3 years) or **Savings Plans** for the baseline, **Azure Hybrid Benefit** for existing Windows/SQL licenses, and **Spot VMs** for interruptible work.

7. **Storage and logs**:

- Lifecycle policies Hot -> Cool -> Cold -> Archive.
- Right redundancy: LRS for non-critical data instead of GRS.
- **Log Analytics**: daily ingestion caps, the Basic logs tier for noisy tables, shorter interactive retention, and only the needed diagnostic categories.

8. **Govern**: **budgets with alerts** per subscription/team, a monthly FinOps review, and cost shown in dashboards so teams see their own numbers.

Always check performance after each change (latency, errors). Saving money must not break SLOs.

Example:

```
az advisor recommendation list --category Cost -o table
az disk list --query "[?diskState=='Unattached'].{name:name,rg:resourceGroup,gb:diskSizeGb}" -o tabl
    e
az network public-ip list --query "[?ipConfiguration==null].{name:name,rg:resourceGroup}" -o table
az vm auto-shutdown -g dev-rg -n dev-vm01 --time 1930
az consumption budget create --budget-name team-a --amount 2000 --time-grain Monthly \
  --category Cost --start-date 2026-10-01 --end-date 2027-09-30
```

> ***What to say to the interviewer:*** *I start with Cost Management and Advisor, with tags enforced so every cost has an owner, then delete orphaned disks, IPs and NICs, rightsize VMs and App Service plans, and add autoscale and AKS spot node pools. Dev/test gets auto-shutdown, the steady baseline gets reservations or savings plans, storage is tiered with lifecycle rules, and Log Analytics ingestion is capped. Budgets and monthly reviews keep it from creeping back, and I verify SLOs*

after each change.

Q32. Security: An image passed all vulnerability scans but still got exploited in production. What did you miss?

In simple words: A scanner is like checking a house against a list of known faulty locks. It can't tell you the window was left open, that a new lock flaw was discovered yesterday, or that the burglar was let in by a guest. A clean scan means "no known bad parts", not "safe".

Detailed answer

Why scans aren't enough: image scanners (Trivy, Grype, ECR/ACR scanning) only match **known CVEs in packages**. They miss **zero-days**, **application logic flaws** (auth bypass, SSRF, injection in your own code), **misconfigurations** and often **secrets**.

Common gaps that let the attack succeed:

1. **Over-privileged pod**: running as root, `privileged: true`, `hostPath` mounts, added capabilities (`NET_ADMIN`, `SYS_ADMIN`), or `hostNetwork`. The attacker can break out to the node.
2. **Default service account token automounted** while the RBAC is too broad. The attacker calls the Kubernetes API, reads secrets or creates pods.
3. **Writable root filesystem**. The attacker downloads and runs a dropper or crypto-miner.
4. **No seccomp/AppArmor profile**, so every syscall is allowed.
5. **Secrets in env vars or image layers**, which the attacker reads straight away.
6. **Open network**: no NetworkPolicy and no egress filtering, so the malware calls back to its C2 server and data leaves freely.
7. **Dependencies pulled at runtime** (`pip install` or `curl | sh` at startup), so the scanned image isn't what actually runs.
8. **Stale scan results**: the image was clean on build day, and a CVE was published later. Rescan the registry and running images continuously.
9. **Unsigned images or mutable tags**: `:latest` was overwritten, so what's running isn't what was scanned.
10. **No runtime detection**: nothing alerted on a shell spawned in a container.

Incident response:

1. **Isolate**: apply a deny-all NetworkPolicy to the pod, cordon the node, and don't delete anything yet.
2. **Capture forensics**: pod logs, `kubectl describe`, a container filesystem snapshot or node disk snapshot, K8s audit logs, cloud audit logs.
3. **Rotate every credential** the pod could reach: SA tokens, DB passwords, cloud keys.
4. **Rebuild** from a clean base, patch the root cause, redeploy with hardened settings, and replace the node.
5. Hold a postmortem and add the missing controls below.

Layer	Control	Tools
Code	SAST, dependency pinning, DAST	SonarQube, Semgrep, OWASP ZAP

Layer	Control	Tools
Build	Image scan, secret scan, minimal/distroless base	Trivy, Grype, Gitleaks
Supply chain	Sign images, deploy by digest, verify at admission	cosign, Kyverno, Binary Authorization
Admission	Restricted Pod Security Standard, policy as code	Pod Security Admission, Gatekeeper, Kyverno
Pod runtime	Non-root, read-only root FS, drop ALL caps, seccomp RuntimeDefault, no SA token	securityContext, automountServiceAccountToken: false
Network	Default-deny ingress and egress, egress allowlist	NetworkPolicy, Cilium/Calico, egress firewall
Detection	Runtime threat detection, continuous rescans	Falco, GuardDuty EKS Runtime, Defender for Containers
Identity	Least-privilege RBAC and workload identity	RBAC, IRSA, Workload Identity

Example:

```
# Hardened pod settings the scan could never have checked
spec:
  automountServiceAccountToken: false
  securityContext:
    runAsNonRoot: true
    seccompProfile:
      type: RuntimeDefault
  containers:
    - name: api
      image: 123456789012.dkr.ecr.ap-south-1.amazonaws.com/api@sha256:3f1c...
      securityContext:
        allowPrivilegeEscalation: false
        readOnlyRootFilesystem: true
        capabilities:
          drop: ["ALL"]

# Contain first: cut the pod's network, keep it for forensics
kubectl label pod api-7d9f compromised=true -n prod
kubectl apply -f deny-all-for-compromised.yaml
kubectl cordon ip-10-0-3-17.ap-south-1.compute.internal
```

> ***What to say to the interviewer:*** *A clean scan only means no known CVEs in packages; it doesn't cover zero-days, app logic flaws, misconfiguration or CVEs published after the scan. The usual gaps are a root or privileged pod with a broad default service account token, a writable filesystem, no seccomp, secrets in env vars, no egress NetworkPolicy, and no runtime detection like Falco or GuardDuty. I'd isolate the pod and node, capture forensics, rotate every reachable credential and rebuild, then close the gaps with signed digests, the restricted Pod Security Standard, default-deny networking and continuous rescans.*